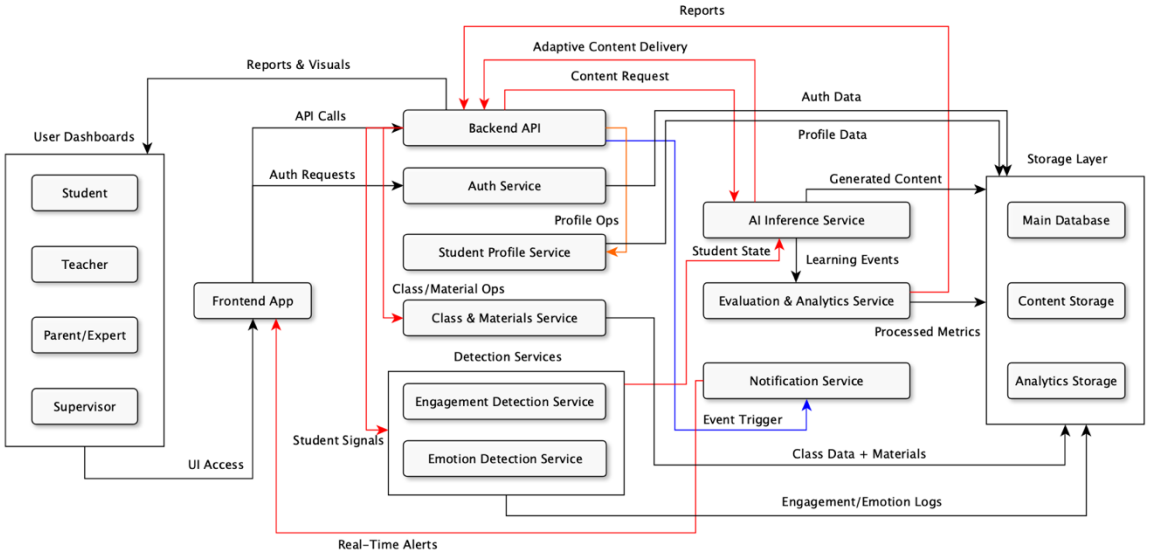
Phase 3

System Analysis & Architecture Package

a) System Architecture / High-Level Framework



1) Major system components

User Roles :

- Student
- Teacher
- Parent / Expert
- Supervisor

Frontend Layer :

- Frontend App (React)

Core Backend / Services :

- Backend API (FastAPI)
- Auth Service
- Student Profile Service
- Class & Materials Service
- Detection Services:
    - Engagement Detection Service
    - Emotion Detection Service

AI & Analytics :

- AI Inference Service
- Evaluation & Analytics Service
- Notification Service

Storage Layer :

- Main Database (PostgreSQL)
- Content Storage (object store / media)
- Analytics Storage (time-series / event logs)

2) Responsibilities

User Roles (Student / Teacher / Parent/Expert / Supervisor) :

- Interact with the system via the Frontend App (role-specific UIs).
- Trigger workflows: account creation, profile edits, class management, content consumption.
- Receive role-appropriate dashboards, alerts, and reports.

Frontend App (React) :

- Single entry point for all users; hosts dashboards and lesson interfaces.
- Calls backend APIs (HTTPS + JWT) and maintains WebSocket connections for live updates.
- Renders adaptive content, notifications, and input capture (answers, audio/video for monitoring).

Backend API (FastAPI) :

- Central orchestrator: exposes REST APIs, routes requests to services and storage.
- Enforces business logic (profiles, classes, gamification, session orchestration).
- Aggregates and routes data between Frontend, AI, Detection, Analytics, and Storage.

Auth Service :

- Handles signup / login, password management, role assignment.
- Issues and validates JWT tokens for stateless authentication and RBAC.
- Provides user identity info to Backend and services.

Student Profile Service :

- Stores and serves student persona data (demographics, preferences, learning-style).
- Exposes profile features for AI personalization and content tailoring.
- Tracks profile edits by teachers/parents and syncs to the main DB.

Class & Materials Service :

- Manages class definitions, milestones, schedules, and teacher uploads.
- Stores content metadata (links) and coordinates content retrieval.
- Provides endpoints for teachers to organize lessons and assign milestones.

Detection Services (Engagement & Emotion) :

- Capture and analyze student interaction streams (clicks, video/audio, activity).
- Produce normalized signals (engagement levels, affective state) and persist them.
- Stream real-time signals to AI and Analytics for adaptive decisions.

AI Inference Service :

- Generates educational content (explanations, quizzes, audio/video segments).
- Produces adaptive lesson plans and runtime adjustments based on student state.
- Consumes profile data, engagement/emotion signals, analytics and teacher/expert inputs.

Evaluation & Analytics Service :

- Processes raw event logs into metrics (mastery, progress, trends).
- Runs batch/near-real-time analytics and produces reports and signals for AI.
- Supplies dashboards and supervisor reports with aggregated insights.

Notification Service :

- Accepts event triggers and pushes real-time notifications via WebSocket / push.
- Delivers adaptive nudges, gamification messages, and important alerts to users.
- Can queue and throttle messages to avoid overload.

Main Database (PostgreSQL):

- Canonical store for users, classes, profiles, gamification points, and metadata.
- Supports transactional operations and relational queries via ORM (SQLModel).

Content Storage (object store / media):

- Stores large binary assets (videos, audio, images, generated content).
- Serves assets to frontend (CDN-friendly) and to AI for post-processing.

Analytics Storage (time-series / event logs):

- Stores time-series or event-stream data (engagement, emotions, raw interactions).
- Optimized for analytics queries used by evaluation service and AI retraining pipelines.

3) Communication protocols

- **Frontend ↔ Backend API:** HTTPS REST (JSON) + **JWT** for auth tokens.
- **Frontend ↔ Notification Service:** WebSocket or Push (real-time updates).
- **Backend ↔ Auth Service:** HTTPS REST (user auth, token introspection).

- **Backend ↔ AI Inference Service:** HTTPS REST (synchronous requests) or gRPC/WebSocket for lower-latency streaming if needed.
- **Backend ↔ Detection Services:** HTTPS REST for batch; WebSocket/streaming for live signals.
- **Services ↔ Storage:** SQL / ORM to Main DB; S3-like API for Content Storage; time-series API for Analytics Storage.
- **Evaluation/Analytics ↔ Backend / Dashboards:** HTTPS REST (report endpoints) and optionally message queues for heavy jobs.
- **Inter-service auth:** Mutual TLS or service tokens (recommended) for service-to-service calls.

4) Architecture summary:

The system is a modular, service-oriented architecture centered on a FastAPI Backend that orchestrates user flows, content management, personalization, and analytics. The React Frontend provides role-specific UIs (student, teacher, parent/expert, supervisor) and communicates with backend services over HTTPS using JWT-based authentication. Core backend services manage profiles, classes, content uploads, and detection pipelines; these services persist canonical data in PostgreSQL and media assets in a content store. An AI Inference Service consumes student profiles, engagement and emotion signals, evaluation results, and teacher/expert inputs to generate and adapt instructional content in real time. Evaluation and analytics services process logs into actionable metrics that feed dashboards and further refine AI personalization.

b) Use Cases & User Scenarios

1. **Student Use Cases**

UC-S1: Access Learning Material

Primary Actor: Student

Stakeholders: Student, Teacher, Parent/Expert

Preconditions:

- Student is authenticated.
- Student has assigned lessons or accessible materials.

Postconditions:

- Selected material is loaded for viewing.

Main Flow:

1. The student logs into the platform.
2. The system displays the student dashboard with available lessons.
3. The student selects a lesson or material.
4. The system loads the content in the adaptive viewer.
5. The student can navigate between pages or sections.

Alternate Flows:

- A1: The student resumes a previously opened material → System opens it at the last saved point.

Exception Flows:

- E1: Material not found → System shows error and suggests other materials.

UC-S2: Start Adaptive Lesson

Primary Actor: Student

Stakeholders: AI Tutor, Teacher

Preconditions:

- Student has opened a lesson.
- Adaptive engine is available.

Postconditions:

- Lesson adapts dynamically to student's emotional and engagement state.

Main Flow:

1. Student starts the lesson.
2. System activates sensors/inputs (interaction, behavior cues, performance).
3. AI Tutor evaluates focus, stress, and engagement.
4. The lesson adjusts pace, difficulty, visuals, and interactions.
5. Student continues learning with real-time adaptation.

Alternate Flows:

- A1: Student switches to “manual mode” → System disables adaptations.

Exception Flows:

- E1: Sensors unavailable → System uses default pacing and difficulty.

UC-S3: Choose Communication Mode (Voice/Text/Touch)

Primary Actor: Student

Stakeholders: AI Tutor

Preconditions:

- Student is inside a lesson or activity.

Postconditions:

- Selected communication mode becomes active.

Main Flow:

1. Student opens the communication menu.
2. Student selects voice, text, or touch mode.
3. System activates the selected mode.
4. Student continues interacting using the chosen method.

Alternate Flows:

- A1: Student changes mode mid-lesson → System switches instantly.

Exception Flows:

- E1: Microphone unavailable for voice → System notifies and suggests text.

UC-S4: Ask AI Tutor for Explanations

Primary Actor: Student

Stakeholders: AI Tutor, Teacher

Preconditions:

- Lesson or content is open.

Postconditions:

- AI Tutor provides explanation.

Main Flow:

1. Student asks a question (voice/text/touch).
2. AI Tutor interprets the query.
3. The system generates a simplified explanation adapted to student's level.
4. Student receives multimodal response (visual + text + optional audio).

Alternate Flows:

- A1: Student requests "simpler explanation" → AI Tutor further simplifies.

Exception Flows:

- E1: AI Tutor cannot interpret question → System asks for clarification.

UC-S5: Take Assessment

Primary Actor: Student

Stakeholders: Teacher, Parent, Supervisor

Preconditions:

- Student completed or started a lesson.
- Assessment is assigned.

Postconditions:

- Score is saved; feedback is generated.

Main Flow:

1. Student opens the assessment.
2. System adapts question format based on student's profile.
3. Student answers questions.
4. System evaluates responses instantly.
5. Student views score with positive reinforcement animations.

Alternate Flows:

- A1: Student pauses assessment → System saves progress.

Exception Flows:

- E1: Assessment data corrupted → System loads backup attempt.

UC-S6: Earn Badges / Rewards

Primary Actor: Student

Preconditions:

- Student completed learning milestones.

Postconditions:

- Badge stored in profile.

Main Flow:

1. Student completes a task or milestone.
2. System checks badge conditions.
3. Badge or reward animation is displayed.
4. Badge appears on the student's dashboard.

Exception Flows:

- E1: Badge service fails → Badge is queued for later.

2. **Teacher Use Cases**

UC-T1: Create & Manage Student Profile

Primary Actor: Teacher

Stakeholders: Student, Parent, Expert

Preconditions:

- Teacher is authenticated and authorized.

Postconditions:

- Student profile is created/updated.

Main Flow:

1. Teacher navigates to the student management section.
2. Teacher selects “Create Profile” or “Edit Profile.”
3. Teacher fills or updates academic, behavioral, and sensory preferences.
4. System validates the inputs.
5. Profile is saved.

Alternate Flows:

- A1: Teacher imports student data → System pre-fills fields.

Exception Flows:

- E1: Missing mandatory fields → System requests completion.

UC-T2: Upload & Manage Learning Materials

Primary Actor: Teacher

Stakeholders: AI Tutor, Student

Preconditions:

- Valid file formats allowed.

Postconditions:

- Material available for assignment.

Main Flow:

1. Teacher selects “Upload Material.”
2. Teacher chooses file(s).
3. System processes the file.
4. AI Tutor converts it into an interactive lesson.
5. Teacher reviews the converted version.

Alternate Flows:

- A1: Teacher rejects AI version → Teacher edits or uploads again.

Exception Flows:

- E1: Unsupported file format → System prompts for alternatives.

UC-T3: Assign Lessons to Student

Primary Actor: Teacher

Stakeholders: Student

Preconditions:

- Lesson is uploaded or available.

Postconditions:

- Student sees assigned lesson.

Main Flow:

1. Teacher opens class or student page.
2. Teacher selects a lesson.
3. Teacher assigns the lesson to the student or entire class.
4. System notifies the student.

Alternate Flows:

- A1: Teacher schedules assignment → System publishes it later.

Exception Flows:

- E1: Student not enrolled → System prevents assignment.

UC-T4: Monitor Real-Time Student Engagement

Primary Actor: Teacher

Stakeholders: Student

Preconditions:

- Student is active in a lesson.

Postconditions:

- Teacher has updated insights.

Main Flow:

1. Teacher opens monitoring dashboard.
2. System shows engagement level (focus, stress, activity).
3. Teacher observes real-time indicators.
4. Teacher may intervene or adjust lesson if needed.

Alternate Flows:

- A1: Teacher switches view to another class → System refreshes data.

Exception Flows:

- E1: Student's signals unavailable → System shows fallback metrics.

UC-T5: View Progress Reports

Primary Actor: Teacher

Stakeholders: Student, Parent

Preconditions:

- Student has historical data.

Postconditions:

- Teacher interprets progress.

Main Flow:

1. Teacher navigates to student reports.
2. System loads academic and emotional trends.
3. Teacher reviews visual insights and summaries.
4. Teacher uses insights to plan next lessons.

Alternate Flows:

- A1: Teacher filters by date or metric → System reloads data.

Exception Flows:

- E1: Report data missing → System displays placeholder.

3. Parent / Expert Use Cases

UC-P1: Create or Update Student Profile

Primary Actor: Parent / Expert

Stakeholders: Teacher, Student

Preconditions:

- Parent/expert has authorized access.

Postconditions:

- Profile updated with preferences or behavioral notes.

Main Flow:

1. Parent/expert opens student profile.
2. They edit sensory preferences, communication mode, or personal data.
3. System validates updates.
4. Changes are saved and sent to AI Tutor for adaptation.

Alternate Flows:

- A1: Parent uploads external reports → System attaches them to profile.

Exception Flows:

- E1: Unauthorized access → System blocks the action.

UC-P2: Monitor Student Progress

Primary Actor: Parent / Expert

Stakeholders: Teacher, Student

Preconditions:

- Student has completed lessons.

Postconditions:

- Parent/expert views understanding of progress.

Main Flow:

1. Parent/expert opens the progress dashboard.
2. System displays performance, behavior, engagement, and improvements.
3. Parent/expert reviews history and trends.
4. Parent/expert optionally downloads the report.

Alternate Flows:

- A1: Parent filters by week/month → System updates view.

Exception Flows:

- E1: No data available → System explains “no recent activity.”

5. Supervisor Use Cases

UC-SV1: View Institution-Level Reports

Primary Actor: Supervisor

Stakeholders: Teachers, Students, Management

Preconditions:

- Supervisor has access rights.

Postconditions:

- Supervisor reviews aggregated insights.

Main Flow:

1. Supervisor opens institutional dashboard.
2. System displays classroom-level and school-level reports.
3. Supervisor reviews academic trends, stress patterns, engagement metrics.
4. Supervisor identifies areas needing support.

Alternate Flows:

- A1: Supervisor filters by grade, class, or time → System updates charts.

Exception Flows:

- E1: Missing data → System shows incomplete sections with warnings.

UC-SV2: Monitor Teachers and Classrooms

Primary Actor: Supervisor

Stakeholders: Teachers

Preconditions:

- Teacher activity data is available.

Postconditions:

- Supervisor receives insights about teacher performance.

Main Flow:

1. Supervisor opens teacher monitoring module.
2. System shows teaching activity, class engagement, and timely content uploads.
3. Supervisor reviews data for performance evaluation.
4. Supervisor may send recommendations or feedback.

Alternate Flows:

- A1: Supervisor views only specific teacher(s) → System filters data.

Exception Flows:

- E1: Teacher activity missing → System marks “no data available.”

UC-SV3: Track Students at Scale

Primary Actor: Supervisor

Stakeholders: Teachers, Students

Preconditions:

- Students have activity logs.

Postconditions:

- Supervisor views institution-wide student trends.

Main Flow:

1. Supervisor opens the student analytics section.
2. System lists all students with key performance indicators.
3. Supervisor identifies outliers or high-risk cases.
4. Supervisor coordinates with teachers for interventions.

Alternate Flows:

- A1: Supervisor exports data → System generates CSV/PDF.

Exception Flows:

- E1: Data export fails → System notifies and suggests retry.

c) Functional Requirements (FR)

User Accounts & Authentication

- **FR-01:** The system shall allow users to register with email, password, and role (Teacher, Supervisor, Parent, Expert, Student).
- **FR-02:** The system shall authenticate users using their email and password.
- **FR-03:** The system shall block login attempts after 5 consecutive failed attempts.
- **FR-04:** The system shall allow users to reset their password via email.

Profile Management

- **FR-05:** The system shall allow each user to create and edit their profile information (name, picture, role-specific fields).
- **FR-06:** The system shall store profile changes immediately and confirm updates.

Classroom Management

- **FR-07:** The system shall allow teachers to create, edit, and delete classrooms.
- **FR-08:** The system shall allow teachers to assign students to a classroom through a searchable modal.
- **FR-09:** The system shall prevent the assignment of a student already assigned to that classroom.

Student Management

- **FR-10:** The system shall provide a student profile page showing progress, assigned materials, and milestones.
- **FR-11:** The system shall allow teachers and supervisors to edit student information.

Materials & Content

- **FR-12:** The system shall allow teachers and experts to upload materials (PDF, video, image, doc).
- **FR-13:** The system shall validate files for size and supported formats before upload.
- **FR-14:** The system shall allow teachers to assign materials to one or more classrooms.
- **FR-15:** The system shall allow students to view assigned materials.

Curriculum & Milestones

- **FR-16:** The system shall allow experts and teachers to create, edit, and delete curriculum milestones.
- **FR-17:** The system shall visually display milestone ordering and dependencies.
- **FR-18:** The system shall prevent publishing a curriculum if required fields are missing.

Reporting & Analytics

- **FR-19:** The system shall generate student progress reports including completed materials, pending tasks, and milestone status.
- **FR-20:** The system shall allow teachers, supervisors, and parents to filter reports by date, classroom, or subject.

- **FR-21:** The system shall allow users to download reports as PDF or CSV.
- **FR-22:** The system shall show classroom-level insights such as engagement trends and material completion rates.

System-Level Requirements

- **FR-23:** The system shall restrict certain actions based on role permissions (Parents can only view; Experts can edit curriculum).
- **FR-24:** The system shall log major actions (material uploads, curriculum edits, classroom creation).

FR-25: The system shall display clear error messages when operations fail (upload errors, missing fields, permission denied).

d) Non-Functional Requirements (NFR)

Performance

- **NFR-01 (Response Time):** The system must load dashboard views in ≤ 3 seconds under normal network conditions
- **NFR-02 (File Upload):** The system must handle file uploads up to 100 MB and process them within ≤ 10 seconds.
- **NFR-03 (Scalability):** The system must support at least 500 simultaneous users without performance degradation.

Reliability & Availability

- **NFR-04 (Uptime):** The system must maintain ≥ 99% availability per month.
- **NFR-05 (Error Recovery):** The system must provide meaningful fallback messages when analytics or storage services are temporarily unavailable.
- **NFR-06 (Data Integrity):** The system must prevent duplicate entries for students, classrooms, and curriculum items.

Security & Privacy

- **NFR-07 (Data Protection):** All user data must be encrypted in transit (HTTPS) and at restز
- **NFR-08 (Access Control):** Users must only access screens and actions allowed by their role.

- **NFR-09 (Sensitive Data):** The system must not expose student personal data through logs, URLs, or downloadable files without authorization.

Usability & Accessibility

- **NFR-11 (Learnability):** A new teacher must be able to complete core tasks (create class, upload material) within 10 minutes without training.
- **NFR-12 (Mobile Support):** The system must be fully usable on devices with screen widths ≥ 360px.

Maintainability

- **NFR-13 (Code Quality):** The system must follow a modular architecture where UI, backend, and analytics components are separated.
- **NFR-14 (Logging):** All server errors must be logged with timestamps and request metadata.
- **NFR-15 (Versioning):** Curriculum and materials must support version tracking to allow rollback.

Compatibility

- **NFR-16:** The system must work on modern browsers: Chrome, Firefox, Edge, Safari (last 2 versions).

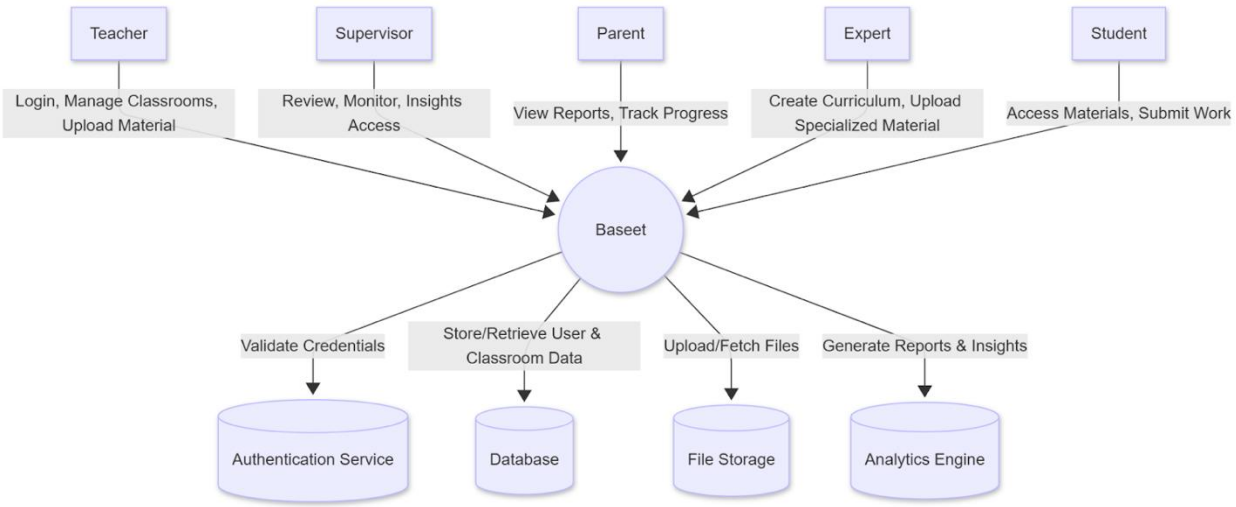
e) Modules

1- **Account Setup Module:**
   - Expert/Parent, Supervisor, and Teacher sign-up and login.
   - Create and open profiles.
   - Teachers can create classrooms and add students.
2- **Student Registration Module:** Register and create student profiles for personalized tracking.
3- **Set Up Content Module:** Upload course materials and create lessons for classroom use.
4- **Personalization Module:** Initialize user personas to tailor content and AI interactions.
5- **Control & Accessibility Module:** Edit profiles and configure user options.
6- **Content Generation Module:** AI-generated explanations, quizzes, videos, audio sequences, and text explanations to support learning.
7- **Gamification Module:**
   - Reward points for correct answers.
   - Deduct points for incorrect answers to motivate engagement.
8- **Student Monitoring Module:** Track student engagement, stress levels, and voice tone during learning.
9- **Adaptive Tutor Module:** Respond to audio interruptions and analyze them to adapt content and guidance in real time.
10- **Evaluation Module:** Evaluate student performance, update AI personalization, and generate reports for the Reporting & Insight Module.
11- **Reporting & Insight Module:**

- Expert, Supervisor, and Teacher dashboards for tracking student progress.
- Central data storage for academic and emotional information.

f) Diagrams

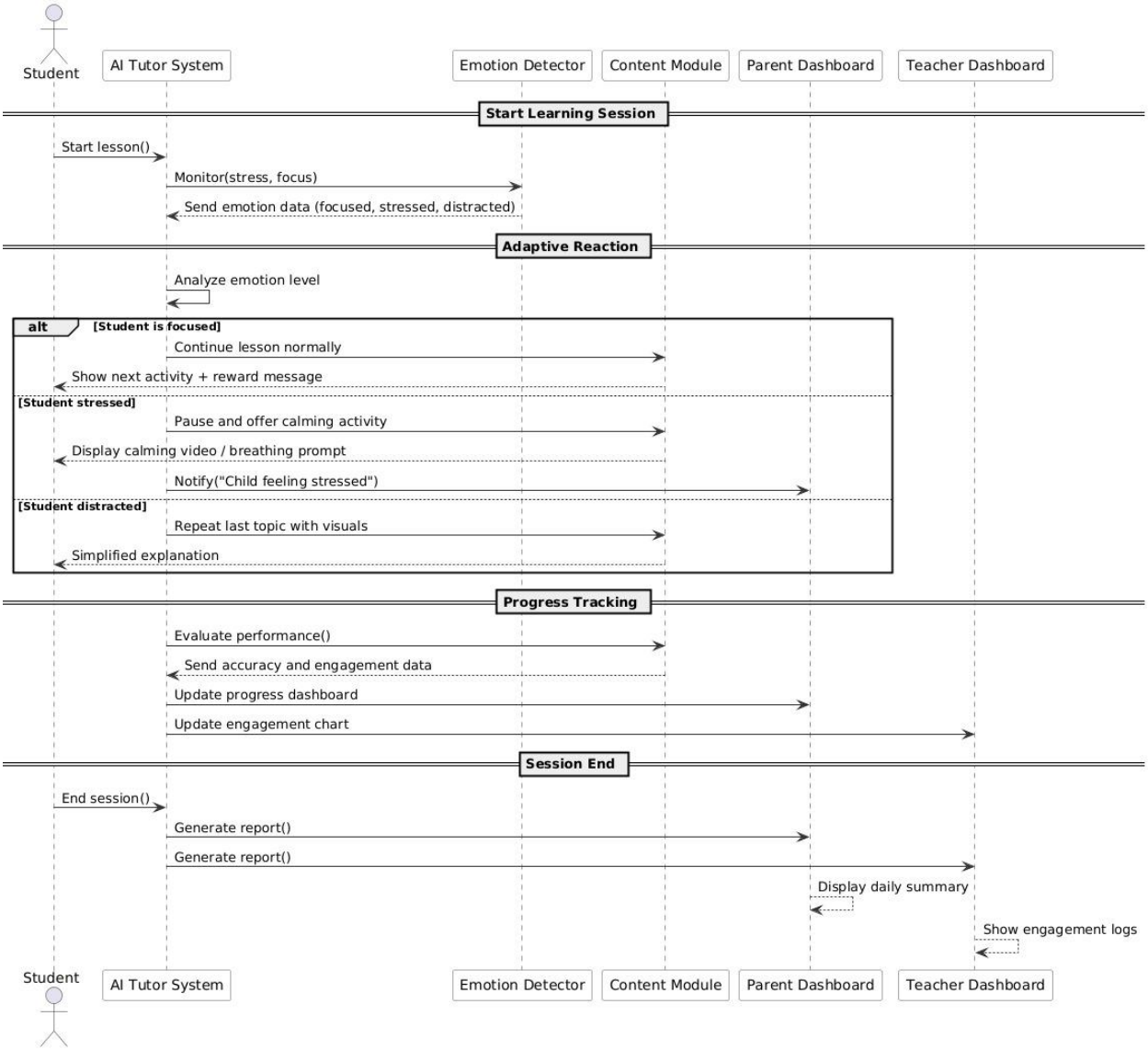
1) System Context Diagram:



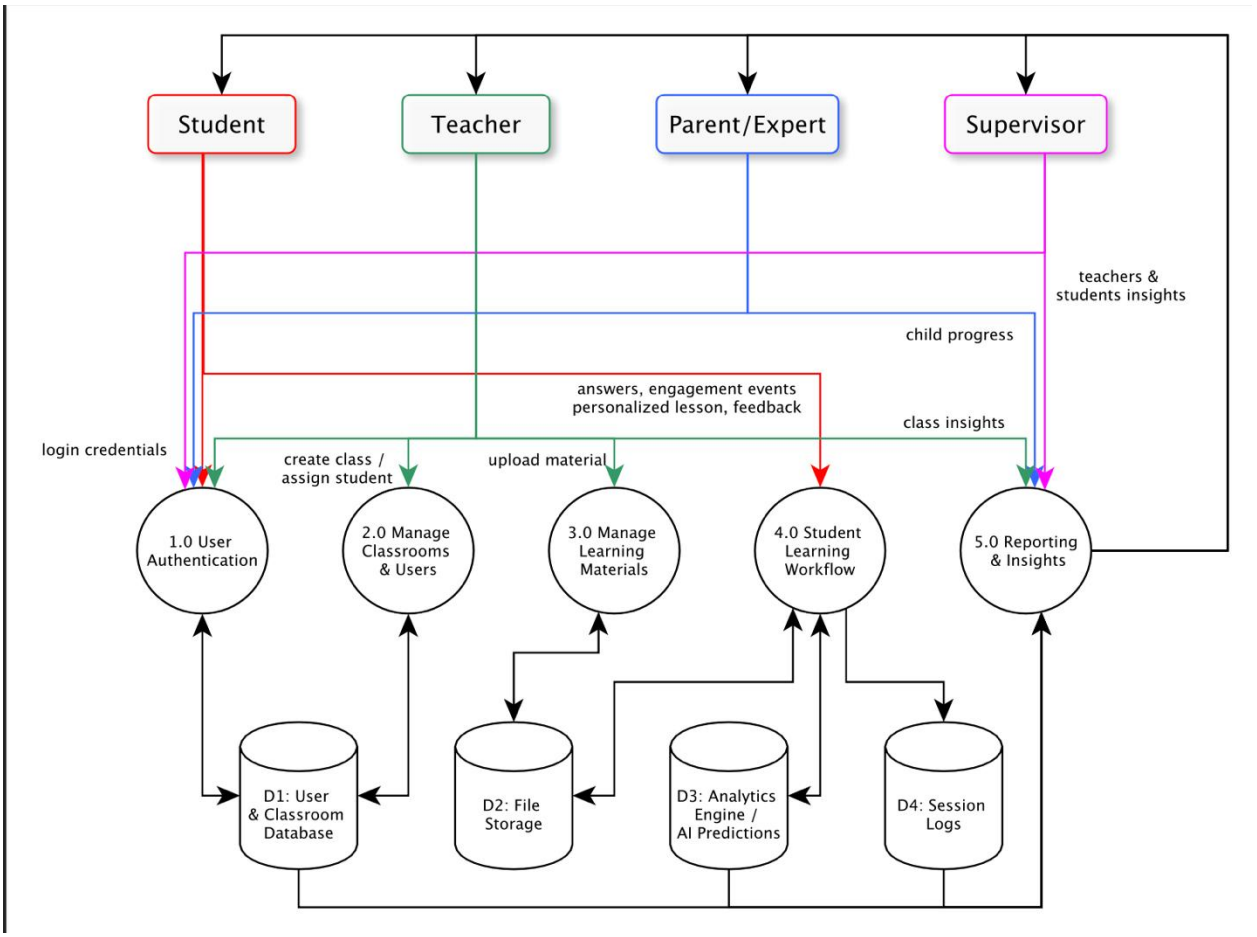
The diagram represents Baseet as the central software platform, surrounded by all external actors who interact with it. Outside the system boundary are five user roles—Teacher, Supervisor, Parent, Expert, and Student—each engaging with the system through specific data flows. Teachers and Experts send materials, curriculum updates, and classroom management actions into the system, while Students send submissions and receive assigned learning content. Supervisors and Parents mainly retrieve analytics and progress information. These arrows represent the essential information exchanged, clearly showing how each persona communicates with the platform without exposing internal processes.

Inside the system boundary are the core backend services that support the platform's functionality. Baseet relies on an Authentication Service for identity verification, a Database for storing structured data such as users, classrooms, curriculum, and progress logs, File Storage for handling large uploaded materials, and an Analytics Engine for generating performance insights. These internal components are not directly visible to the external actors—they are managed by the system itself.

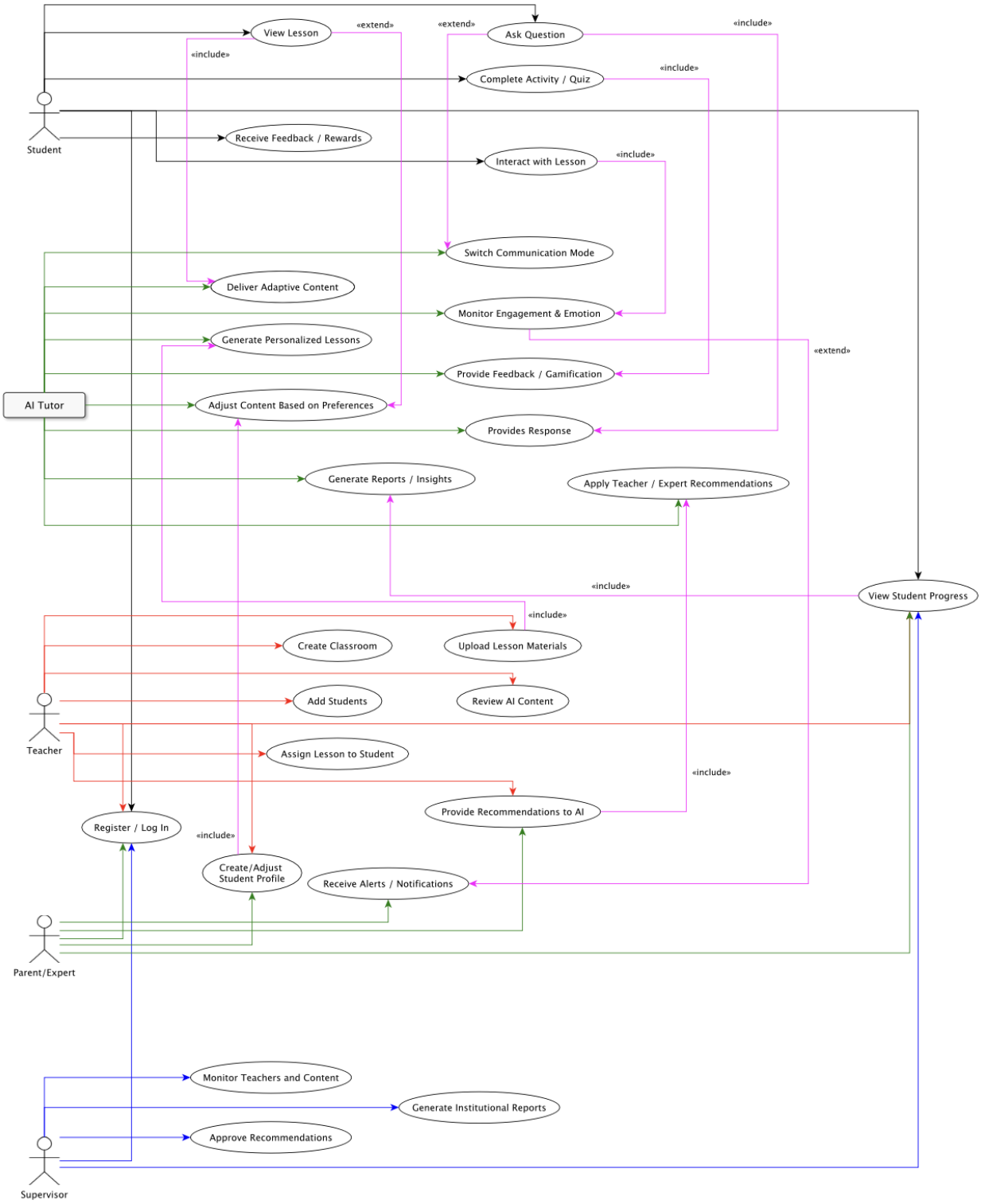
2) Sequence Diagram



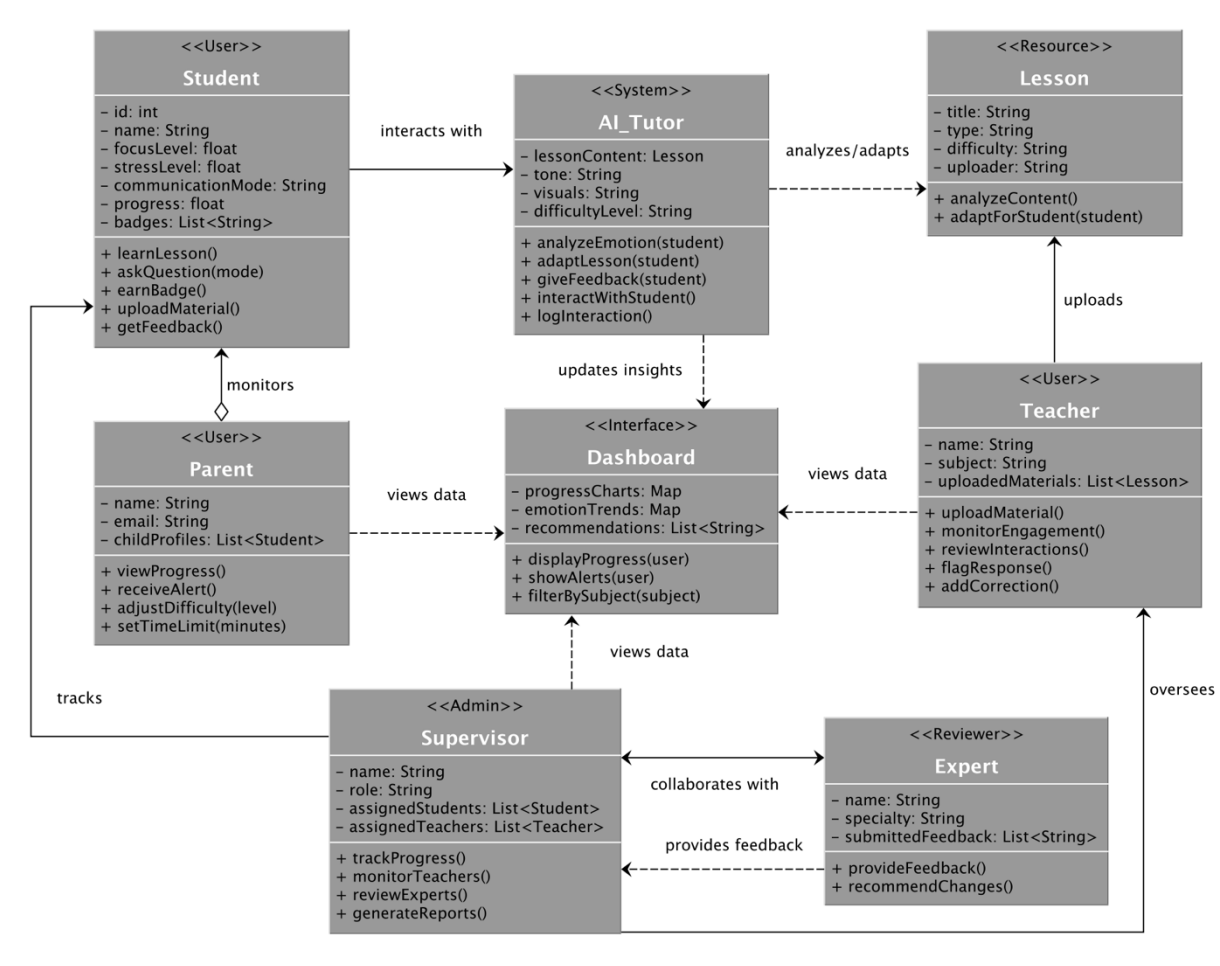
c) Data Flow Diagram (DFD)



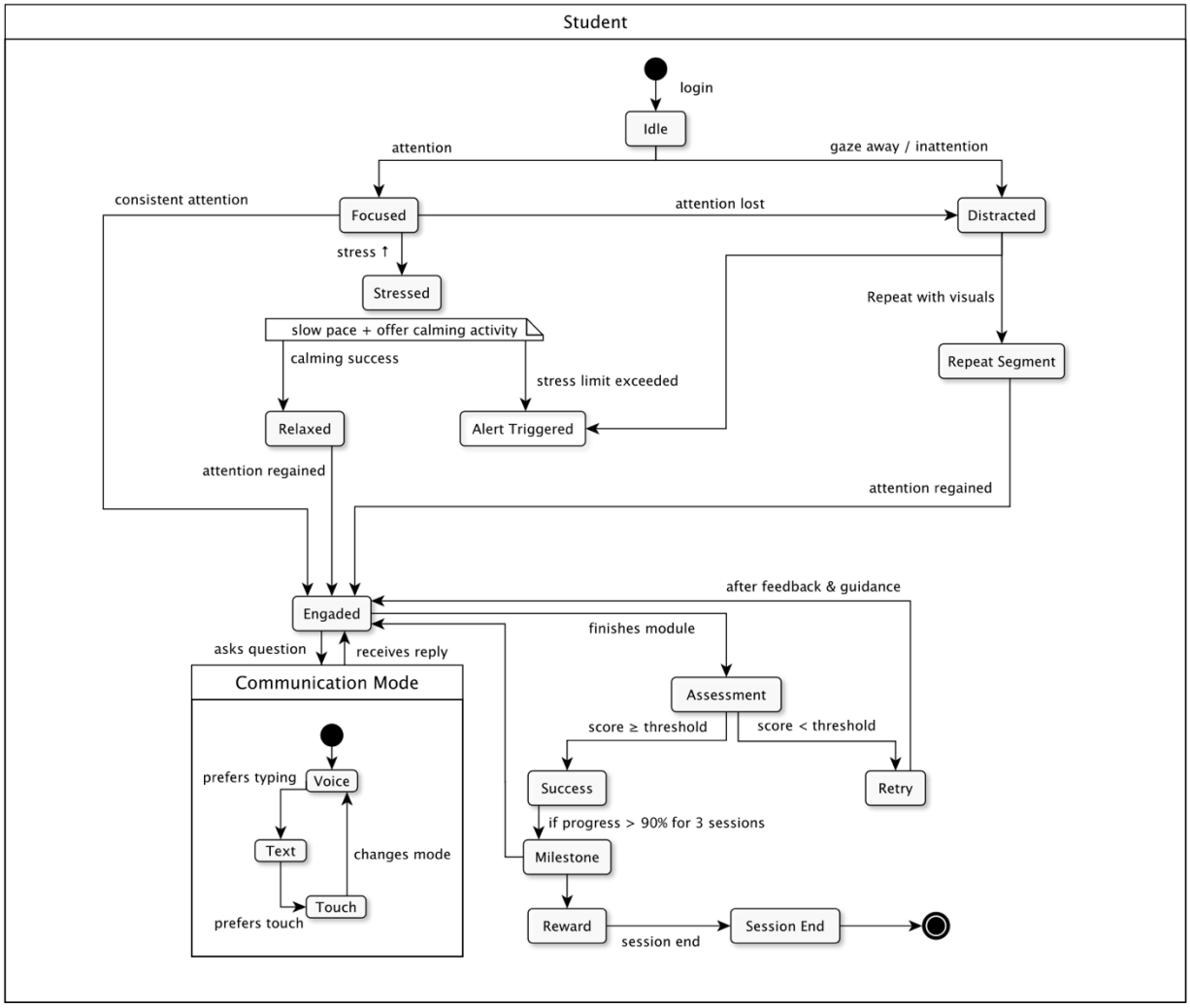
d) Use Case Diagram

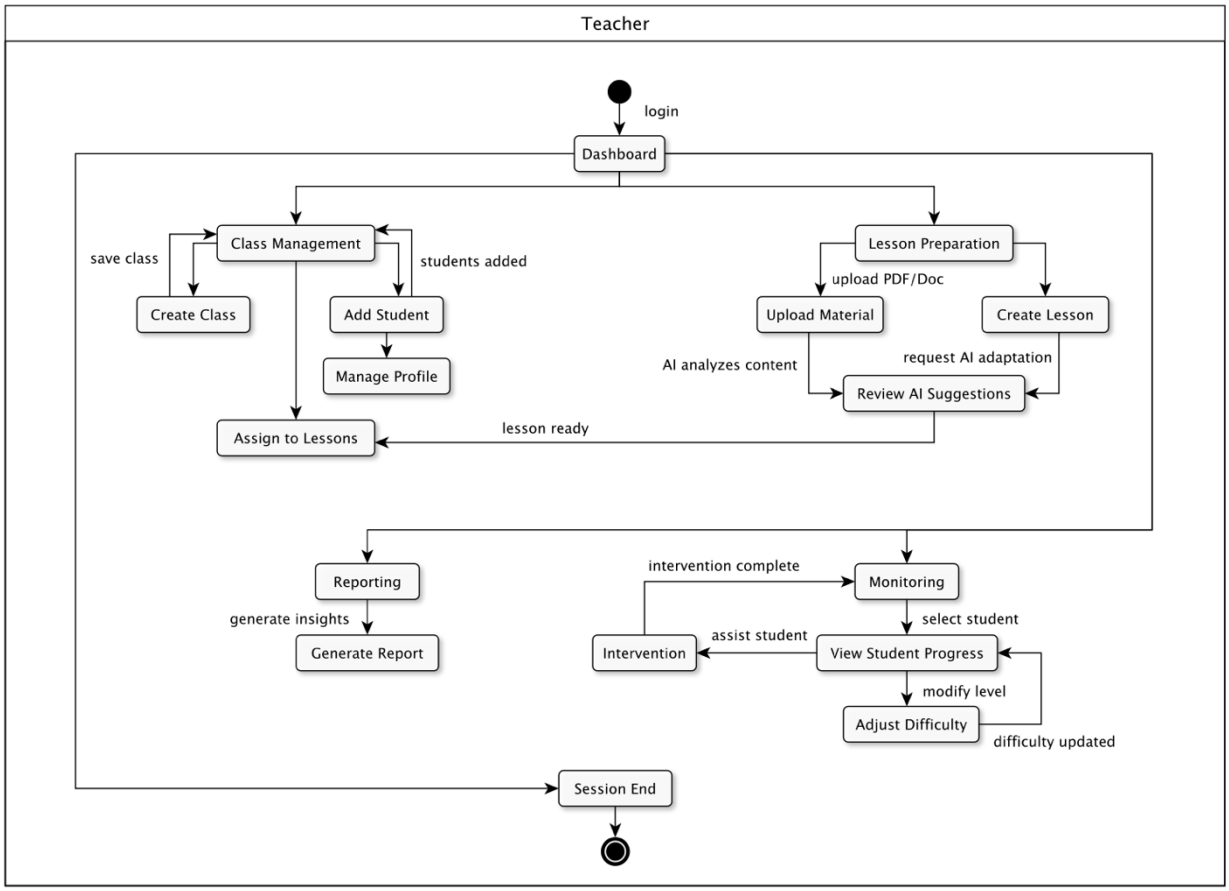


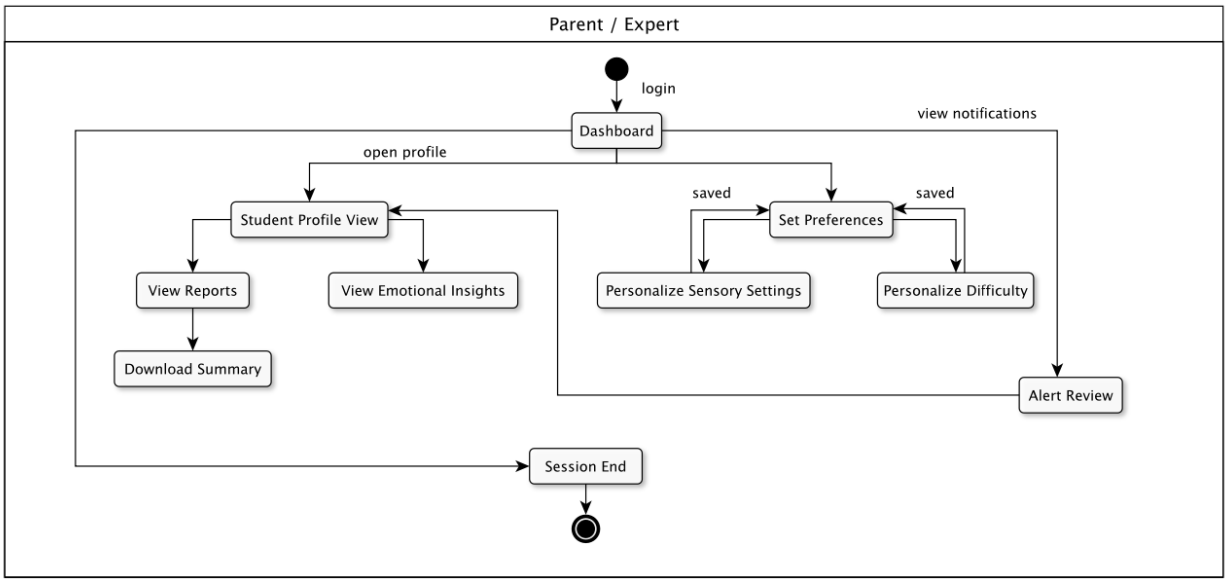
e) Class Diagram

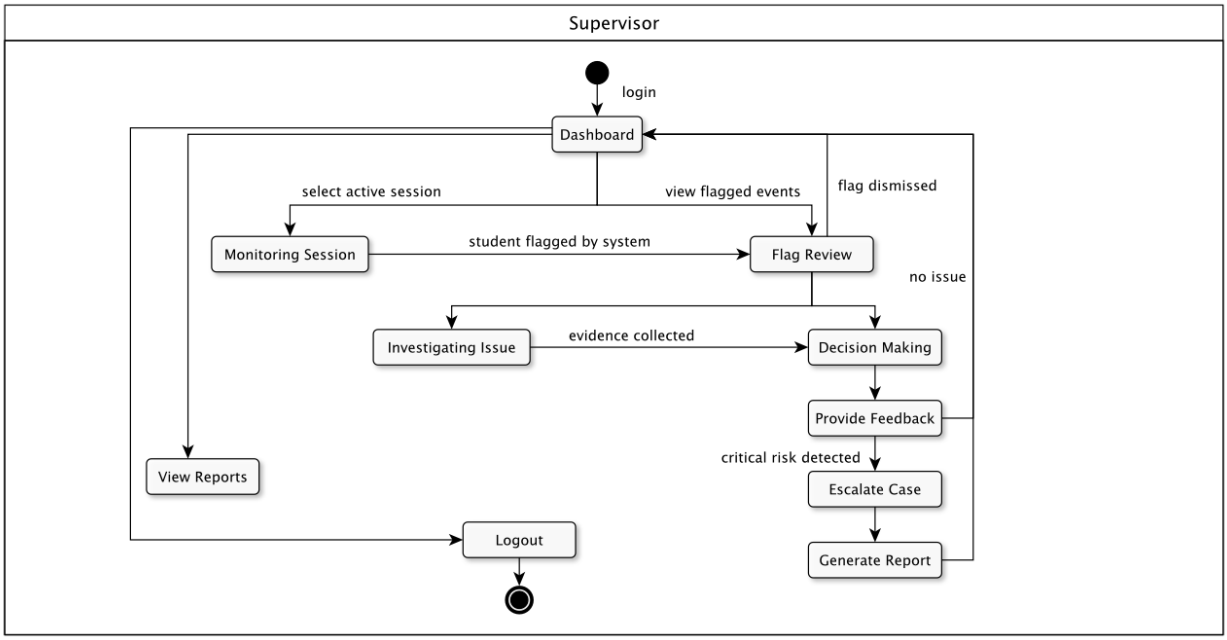


f) State Diagrams




Teacher
login
Dashboard
Class Management
save class
Create Class
students added
Add Student
Manage Profile
Assign to Lessons
Lesson Preparation
upload PDF/Doc
Upload Material
Create Lesson
AI analyzes content
request AI adaptation
Review AI Suggestions
lesson ready
Reporting
generate insights
Generate Report
intervention complete
Monitoring
select student
Intervention
assist student
View Student Progress
modify level
Adjust Difficulty
difficulty updated
Session End


Parent / Expert
login
Dashboard
view notifications
open profile
saved
saved
Student Profile View
Set Preferences
View Reports
View Emotional Insights
Personalize Sensory Settings
Personalize Difficulty
Download Summary
Alert Review
Session End


Supervisor
login
Dashboard
select active session
view flagged events
flag dismissed
Monitoring Session
student flagged by system
Flag Review
no issue
Investigating Issue
evidence collected
Decision Making
Provide Feedback
critical risk detected
View Reports
Escalate Case
Logout
Generate Report

g) Activity Diagram